E11 — Maximum Depth of Binary Tree (DFS / BFS)

Experiment: 11 | **LeetCode:** #104 | **CO:** CO2, CO3 | **BT Level:** BT5

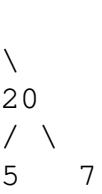
Problem Statement

Given the `root` of a binary tree, return its **maximum depth**.

The maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example 1:

```
Input:  root = [3,9,20,null,null,15,7]
Output: 3
```



Example 2:

```
Input:  root = [1,null,2]
Output: 2
```

Constraints:

- The number of nodes is in the range `[0, 10^4]`
- `-100 ≤ Node.val ≤ 100`

Concept: Maximum Depth of a Binary Tree

The **depth** of a node is the number of edges from the root to that node.
The **height** (maximum depth) of a tree = max depth among all leaf nodes.

Key insight (recursive):

> The height of a tree = 1 + max(height of left subtree, height of right subtree)

Base case: An empty tree (None) has height 0.

```
height(3) = 1 + max(height(9), height(20))
          = 1 + max(1, 2)
          = 3
```

Solution 1: Recursive DFS (Post-order)

At each node, recursively compute the depth of left and right subtrees, then return `1 + max(left, right)`.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def maxDepth_dfs(root):
    """
    Recursive DFS (Post-order traversal).
    Time: O(n)  |  Space: O(h)  where h = height of tree
    Worst case space: O(n) for skewed tree, O(log n) for balanced tree.
    """
    if root is None:
        return 0
    left_depth  = maxDepth_dfs(root.left)
    right_depth = maxDepth_dfs(root.right)
    return 1 + max(left_depth, right_depth)
```

Solution 2: Iterative BFS (Level Order)

Use a queue and count the number of levels processed. Each level = one depth unit.

```
from collections import deque

def maxDepth_bfs(root):
    """
    Iterative BFS — level-by-level traversal.
    Time: O(n)  |  Space: O(w)  where w = max width of tree
    """
    if root is None:
        return 0

    queue = deque([root])
    depth = 0

    while queue:
        depth += 1
        # Process all nodes at the current level
        for _ in range(len(queue)):
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
```

```
    return depth
```

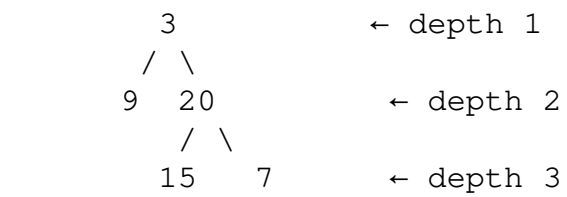
Solution 3: Iterative DFS (Explicit Stack)

Simulate the recursive call stack using an explicit stack storing `(node, current_depth)` pairs.

```
def maxDepth_iterative(root):
    """
    Iterative DFS with explicit stack.
    Time: O(n)  |  Space: O(h)
    """
    if root is None:
        return 0

    stack = [(root, 1)]
    max_depth = 0

    while stack:
        node, depth = stack.pop()
        max_depth = max(max_depth, depth)

        if node.left:
            stack.append((node.left, depth + 1))
        if node.right:
            stack.append((node.right, depth + 1))

    return max_depth
```

Detailed Trace

Tree: `[3, 9, 20, null, null, 15, 7]`



Recursive DFS trace:

```
maxDepth(3)
  maxDepth(9)
    maxDepth(None) → 0
    maxDepth(None) → 0
    return 1 + max(0, 0) = 1
  maxDepth(20)
    maxDepth(15)
      maxDepth(None) → 0
      maxDepth(None) → 0
```

```
      return 1 + max(0, 0) = 1
    maxDepth(7)
      maxDepth(None) → 0
      maxDepth(None) → 0
      return 1 + max(0, 0) = 1
    return 1 + max(1, 1) = 2
  return 1 + max(1, 2) = 3 ✓
```

BFS trace:

```
Queue: [3]              depth = 1
  Process 3 → enqueue 9, 20
Queue: [9, 20]          depth = 2
  Process 9 → no children
  Process 20 → enqueue 15, 7
Queue: [15, 7]          depth = 3
  Process 15 → no children
  Process 7 → no children
Queue: []

return depth = 3 ✓
```

Complexity Summary

Approach	Time	Space
Recursive DFS	O(n)	O(h)
Iterative BFS	O(n)	O(w) — max width
Iterative DFS	O(n)	O(h)

Where `h` = height of tree, `w` = max width of tree.
For a **balanced** tree: h = O(log n), w = O(n/2) ≈ O(n).
For a **skewed** tree: h = O(n), w = O(1).

Edge Cases

```
# Empty tree
maxDepth_dfs(None)    # → 0

# Single node
root = TreeNode(1)
maxDepth_dfs(root)    # → 1

# Skewed tree (like a linked list)
# 1 → 2 → 3 → 4 → 5
maxDepth_dfs(root)    # → 5
```

Key Takeaways

- **Recursive DFS** is the most concise — direct application of the tree's recursive structure.
- **BFS** naturally counts levels; useful when you also need level-wise data.
- **Iterative DFS** avoids Python's recursion limit for very deep trees.
- Maximum depth = minimum number of steps to reach the deepest leaf.